

งานปฏิบัติการ: Multi-threaded File Download Client/Server

Protocol, Concurrency และการเปรียบเทียบ Traditional I/O กับ NIO Native Transfer

วัตถุประสงค์

- สร้าง Client/Server สำหรับดาวน์โหลดไฟล์แบบหลายส่วนพร้อมกัน
- ออกแบบ protocol สำหรับการขอรายชื่อไฟล์ ขอข้อมูลไฟล์ และรับส่งข้อมูลแต่ละช่วงอย่างถูกต้อง
- เปรียบเทียบ Traditional I/O กับ NIO/native-transfer path และอธิบายผลด้านเวลา/throughput

ข้อกำหนดของโปรแกรม

- Client และ Server ทำงานผ่าน TCP ใช้ command-line ได้ ไม่บังคับ GUI
- Server ต้องรองรับหลาย connection พร้อมกัน โดยเลือกใช้ Thread, Thread Pool หรือ Virtual Thread ได้
- Client ต้องแบ่งไฟล์เป็น 10 ช่วง และใช้ 10 workers พร้อมกัน โดยแต่ละ worker มี connection, offset และ length ของตนเอง และช่วงข้อมูลต้องไม่ซ้อนกัน
- การเขียนไฟล์ปลายทางเลือกได้ 2 แบบ: เขียนเป็น part files แล้ว merge หรือเขียนลง final file ตาม offset ด้วย RandomAccessFile/FileChannel (ควรแยก file handle/channel ของแต่ละ worker หรือใช้ positional write อย่างถูกต้อง)

Protocol ที่ต้องออกแบบ

นักศึกษากำหนดรูปแบบ request/response เอง แต่ต้องรองรับอย่างน้อยดังนี้

คำสั่งตัวอย่าง	หน้าที่	ตัวอย่างผลตอบกลับ
LIST	ขอรายชื่อไฟล์ที่ดาวน์โหลดได้	FILE <name> <size> ...
INFO <filename>	ขอข้อมูลไฟล์ก่อนเริ่มดาวน์โหลด	SIZE <bytes> หรือ ERROR
GET <filename> <offset> <length>	ขอข้อมูลเฉพาะช่วงของไฟล์	ส่ง payload ตามจำนวน byte ที่ตกลงกัน
ERROR <code> <message>	แจ้ง request หรือช่วงข้อมูลไม่ถูกต้อง	ข้อความที่ Client นำไปจัดการได้

แนวทางการทำงาน

1. Client ขอรายชื่อ/ข้อมูลไฟล์และขนาดไฟล์จาก Server
2. Client คำนวณ byte range สำหรับ 10 workers; worker สุ่มทำรับช่วงที่เหลือ
3. แต่ละ worker เปิด connection ของตนเอง ส่ง GET พร้อม offset/length และรับข้อมูลเฉพาะช่วงนั้น
4. Server handler อ่านเฉพาะช่วงที่ร้องขอ; เมื่อครบทุก worker ให้ Client merge part files หรือยืนยันตำแหน่งที่เขียน แล้วตรวจ size/hash

Method/API ที่แนะนำ (ไม่บังคับใช้ทั้งหมด)

หัวข้อ	ตัวอย่าง API/Method
Network	ServerSocket.accept(), Socket, SocketChannel
Concurrency	Thread, Executors.newFixedThreadPool(...), Executors.newVirtualThreadPerTaskExecutor() (Java 21+)
Traditional I/O	InputStream.read(...), OutputStream.write(...)
แบ่ง/เขียนช่วงไฟล์	RandomAccessFile.seek(...), FileChannel.position(...), FileChannel.write(buffer, position)
NIO native transfer	FileChannel.transferTo(...), FileChannel.transferFrom(...)

การทดลองและการวัดผล

- ใช้ไฟล์เดียวกันและสภาพแวดล้อมเดียวกัน เปรียบเทียบ Traditional I/O กับ NIO/native transfer
- อย่างน้อยทดสอบ 1 worker และ 10 workers สำหรับแต่ละวิธี และรันแต่ละกรณีอย่างน้อย 3 ครั้ง
- บันทึกเวลารวมและ throughput (MB/s) พร้อมตรวจ size/hash; อธิบายเหตุผลที่ 10 workers อาจไม่เร็วขึ้น 10 เท่า และ NIO อาจไม่ชนะทุกครั้ง
- ถ้าทดสอบผ่าน localhost ให้ระบุข้อจำกัดจาก Page Cache, loopback network และ storage cache

สิ่งที่ต้องส่ง

- Source code ของ Client และ Server พร้อมคำสั่ง compile/run